# Daily-Brew Group Project

Daily-Brew is our group ETL project for processing cafe transaction data. The aim of the project is to take raw cafe transaction data, clean it, remove sensitive customer details, store the cleaned data in AWS, and use dashboards to make the results easier to understand.

As a team, we built a cloud-based data pipeline that takes a raw CSV file from Amazon S3, processes it with AWS Lambda, loads the cleaned data into Amazon Redshift, and then displays useful sales and monitoring information in Grafana.

## What the Project Does

The raw cafe CSV file was not ready to use straight away, so our ETL process cleans and reshapes the data before storing it.

The pipeline:

1. Reads the raw CSV file from S3.
2. Adds column names because the original file has no headers.
3. Removes sensitive customer information, including customer names and card numbers.
4. Splits the transaction date and time into separate fields.
5. Converts text fields to lowercase so the data is more consistent.
6. Creates unique IDs for transactions and transaction items.
7. Splits purchased items into separate rows.
8. Loads the cleaned data into Amazon Redshift.
9. Uses Grafana dashboards to visualise the data.

## AWS Pipeline

The main ETL process runs in AWS. When a raw CSV file is uploaded to S3, it triggers a Lambda function. The Lambda function reads the file, cleans and transforms the data, and then loads it into Amazon Redshift.

Cloud pipeline:

```
CSV uploaded to S3
-> S3 triggers Lambda
-> Lambda reads the CSV
-> Lambda removes sensitive data and transforms the rows
-> Lambda loads the cleaned data into Redshift
-> Grafana displays sales and infrastructure dashboards
```

AWS resources used:

- S3 bucket: `daily-brew-group-raw-csv`
- Lambda function: `daily-brew-group-etl-lambda`
- Redshift database: `daily_brew_cafe_db`
- Redshift schema: `daily_brew_group`

Redshift tables:

- `daily_brew_group.transactions`
- `daily_brew_group.transaction_items`

## Data Model

### daily_brew_group.transactions

This table stores one row per transaction.

Columns:

- `transaction_id`
- `branch`
- `total_amount`
- `payment_method`
- `transaction_date`
- `transaction_time`

### daily_brew_group.transaction_items

This table stores one row for each item bought in a transaction.

Columns:

- `transaction_item_id`
- `transaction_id`
- `item_name`
- `item_price`

One transaction can have multiple transaction items.

## Main AWS Files

### aws/lambda/lambda_function.py

This is the Lambda function used for the ETL pipeline.

It:

- receives the S3 event
- gets the bucket name and file name
- loads the CSV from S3 using `boto3`
- removes customer name and card number
- splits transaction date and time
- lowercases text fields
- splits purchased items into separate rows
- generates unique IDs
- inserts the cleaned rows into Redshift

### aws/cloudformation/deployment-bucket-stack.yml

Creates the deployment bucket used for CloudFormation packaging. This bucket is only used for deployment files, not the raw cafe CSV data.

### aws/cloudformation/etl-stack.yml

Creates the main ETL resources:

- raw CSV S3 bucket

- Lambda function

- S3 trigger

- permissions for S3 to trigger Lambda

### aws/scripts/deploy.ps1

PowerShell script used to deploy the ETL CloudFormation stacks.

## Redshift Setup

A separate schema was created for the project:

```
CREATE SCHEMA IF NOT EXISTS daily_brew_group;
```

The tables were created in the `daily_brew_group` schema:

```
CREATE TABLE IF NOT EXISTS daily_brew_group.transactions (
    transaction_id VARCHAR(36) PRIMARY KEY,
    branch VARCHAR(100),
    total_amount DECIMAL(10, 2),
    payment_method VARCHAR(50),
    transaction_date DATE,
    transaction_time TIME
);

CREATE TABLE IF NOT EXISTS daily_brew_group.transaction_items (
    transaction_item_id VARCHAR(36) PRIMARY KEY,
    transaction_id VARCHAR(36),
    item_name VARCHAR(255),
    item_price DECIMAL(10, 2)
);
```

## Test Result

After uploading the raw CSV to `daily-brew-group-raw-csv`, the Lambda successfully loaded:

- 382 transaction rows

- 1051 transaction item rows

Example CloudWatch log output:

```
raw_rows_loaded=382
transactions_inserted=382
transaction_items_inserted=1051
lambda_handler: finished
```

## Useful Redshift Queries

Check row counts:

```
SELECT COUNT(*) AS transaction_count
FROM daily_brew_group.transactions;

SELECT COUNT(*) AS transaction_item_count
FROM daily_brew_group.transaction_items;
```

Total sales:

```
SELECT SUM(total_amount) AS total_sales
FROM daily_brew_group.transactions;
```

Sales by payment method:

```
SELECT payment_method, SUM(total_amount) AS total_sales
FROM daily_brew_group.transactions
GROUP BY payment_method
ORDER BY total_sales DESC;
```

Best-selling items:

```
SELECT item_name, COUNT(*) AS times_sold
FROM daily_brew_group.transaction_items
GROUP BY item_name
ORDER BY times_sold DESC;
```

Revenue by item:

```
SELECT item_name, SUM(item_price) AS item_revenue
FROM daily_brew_group.transaction_items
GROUP BY item_name
ORDER BY item_revenue DESC;
```

Sales by hour:

```
SELECT EXTRACT(hour FROM transaction_time) AS hour_of_day,
       SUM(total_amount) AS total_sales
FROM daily_brew_group.transactions
GROUP BY hour_of_day
ORDER BY hour_of_day;
```

## Grafana Dashboards

We used Grafana to create dashboards from the cleaned Redshift data and AWS monitoring data.

Dashboard flow:

```
Redshift + CloudWatch
-> Grafana running on EC2
-> sales and infrastructure dashboards
```

AWS resources:

- CloudFormation stack: `daily-brew-group-grafana-stack`

- EC2 instance: `daily-brew-group-grafana-ec2`

- Grafana URL: `http://34.250.20.142`

Grafana data sources:

- Daily Brew Redshift, using the PostgreSQL data source to query Redshift

- CloudWatch, used for Lambda and EC2 metrics

Grafana dashboards:

- Daily Brew Sales Dashboard

- Daily Brew Infrastructure Monitoring

The sales dashboard shows:

- total revenue
- total transactions
- best-selling products
- revenue by hour
- revenue by branch
- products sold per branch

The infrastructure dashboard shows:

- Lambda invocations
- Lambda errors
- Lambda duration
- Grafana EC2 CPU utilisation

Dashboard JSON backups are saved in:

`aws/grafana/dashboards/`

## Sensitive Data

The raw CSV contains customer details, so part of the project was making sure that information was not stored in the final tables.

The ETL process does not store:

- customer name
- card number

Only the cleaned transaction and item data is stored in Redshift.

## Group Project Summary

Daily-Brew shows how raw cafe transaction data can be cleaned, protected, stored, and visualised using AWS and Grafana. As a group, we built a full ETL pipeline that removes sensitive data, organises the transactions into useful tables, and provides dashboards for both sales insights and infrastructure monitoring.